

MODULE 04

Enums, Narrowing & Type Guards

Unions give you flexibility. Narrowing gives you control. This module teaches you how TypeScript intelligently tracks what a variable CAN be at each point in your code — and how to use that to write safer React components.

~50 min read

Builds on Module 1 & 3

Critical for React state

1. Enums — Named Constants

An enum (short for enumeration) is a way to define a set of named constants. If you've ever used a plain JS object to group related constants, an enum is the TypeScript version — with type checking built in.

Numeric Enums (default)

Numeric enum

```
// Without enum — error-prone JS approach
const STATUS_PENDING = 0;
const STATUS_ACTIVE = 1;
const STATUS_INACTIVE = 2;

// With enum — TypeScript's named constants
enum Status {
  Pending,    // = 0 (auto-assigned)
  Active,     // = 1
  Inactive,   // = 2
}

let userStatus: Status = Status.Active;
userStatus = Status.Inactive; // OK
userStatus = 5;                // Error: 5 is not a valid Status

console.log(Status.Active);    // 1
console.log(Status[1]);        // 'Active' (reverse lookup!)
```

String Enums (recommended)

String enums are more readable and are what you'll see most often in real codebases. Values are explicit strings instead of numbers:

String enum

```
enum Direction {
  North = 'NORTH',
  South = 'SOUTH',
  East  = 'EAST',
  West  = 'WEST',
}

function move(dir: Direction): void {
  console.log(`Moving ${dir}`);
}

move(Direction.North); // 'Moving NORTH'
move('NORTH');          // Error: string not assignable to Direction
move(Direction.Up);     // Error: 'Up' does not exist on Direction
```

Enum vs Union type — which to use?

For simple fixed sets of strings, many TS developers prefer union types (type Status = 'pending' | 'active' | 'inactive') over enums — they're lighter, need no import, and produce cleaner compiled JS. Use enums when you need reverse lookup, when the values are numeric, or when you're working with a team that prefers the explicitness of enums.

2. What Is Type Narrowing?

When you have a union type like `string | number`, TypeScript doesn't know WHICH one it is. Narrowing is how you prove to TypeScript which specific type you're working with inside a block of code.

Think of it as TypeScript watching your if-statements and getting smarter as it reads your code:

Narrowing with `typeof`

```
function printLength(value: string | number): void {
  // Here: value is string | number — TS doesn't know which
  value.toUpperCase(); // Error! .toUpperCase doesn't exist on number
  value.toFixed(2);    // Error! .toFixed doesn't exist on string

  if (typeof value === 'string') {
```

```
// Here: TS KNOWS value is a string
console.log(value.toUpperCase()); // OK!
console.log(value.length);        // OK!
} else {
  // Here: TS KNOWS value is a number (process of elimination)
  console.log(value.toFixed(2));   // OK!
}
}
```

TypeScript's Control Flow Analysis

TypeScript reads your code's logic and tracks the type of each variable at every point. After a `typeof` check, after a null check, after an `if/else` — TS updates what it knows. This is called control flow analysis and it's what makes TS feel so smart in your editor.

3. `typeof` Type Guard

The `typeof` operator is the most common narrowing tool. It works on JavaScript's primitive types:

```
typeof guard with multiple types
function processInput(input: string | number | boolean): string {
  if (typeof input === 'string') {
    return input.toUpperCase(); // input: string
  } else if (typeof input === 'number') {
    return input.toFixed(2); // input: number
  } else {
    return input ? 'yes' : 'no'; // input: boolean
  }
}

// typeof works with: 'string' | 'number' | 'boolean' | 'object'
//                    'undefined' | 'function' | 'symbol' | 'bigint'
```

4. Truthiness Narrowing — Checking for null/undefined

You already do this in JavaScript — checking if something is truthy before using it. TypeScript understands this and narrows the type accordingly:

Truthiness narrowing

```
function greet(name: string | null | undefined): string {
  // Before check: name is string | null | undefined

  if (!name) {
    return 'Hello, guest!'; // name is null | undefined | ''
  }

  // After check: TS knows name is a non-empty string
  return `Hello, ${name.toUpperCase()}!`; // Safe to call string methods
}

// Common React pattern – optional prop check
function renderAvatar(imageUrl?: string) {
  if (imageUrl) {
    // imageUrl is string here (not undefined)
    return `<img src='${imageUrl}' />`;
  }
  return `<img src='/default.png' />`;
}
```

5. instanceof Type Guard — For Classes & Objects

When working with class instances (like Error objects), use instanceof to narrow the type:

```
instanceof guard
class HttpError extends Error {
  constructor(public statusCode: number, message: string) {
    super(message);
  }
}

class NetworkError extends Error {
  constructor(public url: string, message: string) {
    super(message);
  }
}

function handleError(err: HttpError | NetworkError): void {
  if (err instanceof HttpError) {
    // err is HttpError here
    console.log(`HTTP ${err.statusCode}: ${err.message}`);
  } else {
    // err is NetworkError here
  }
}
```

```
    console.log(`Network error at ${err.url}: ${err.message}`);  
  }  
}
```

6. The 'in' Operator — Narrowing by Property

When you have a union of object types (not classes), use the in operator to check if a property exists. This narrows to the type that has that property:

```
'in' operator narrowing  
interface Cat { meow(): void; }  
interface Dog { bark(): void; }  
  
function makeSound(animal: Cat | Dog): void {  
  if ('meow' in animal) {  
    animal.meow(); // animal is Cat here  
  } else {  
    animal.bark(); // animal is Dog here  
  }  
}  
  
// Very common pattern with API response shapes  
interface SuccessResponse { data: unknown; }  
interface ErrorResponse { error: string; }  
  
function handleResponse(res: SuccessResponse | ErrorResponse): void {  
  if ('error' in res) {  
    console.error(res.error); // res is ErrorResponse  
  } else {  
    console.log(res.data); // res is SuccessResponse  
  }  
}
```

7. Custom Type Guards — is Keyword

Sometimes typeof and instanceof aren't enough. You can write your own type guard function — a function that tells TypeScript 'if this returns true, the argument is definitely this type':

```
Custom type guard with 'is'
```

```
interface User { id: number; name: string; role: 'user'; }
interface Admin { id: number; name: string; role: 'admin'; permissions:
string[]; }

// Custom type guard – note the 'is' keyword in the return type
//
function isAdmin(user: User | Admin): user is Admin {
  return user.role === 'admin';
}

function grantAccess(user: User | Admin): void {
  if (isAdmin(user)) {
    // user is Admin here – TS knows!
    console.log(user.permissions); // OK – Admin has permissions
  } else {
    // user is User here
    console.log('Read-only access');
  }
}
```

8. Discriminated Unions — The Most Powerful Pattern

This is the pattern that makes complex state management in React bulletproof. A discriminated union is a union of objects that all share one common property (the 'discriminant') with a unique literal type. TypeScript uses that property to narrow automatically:

Discriminated union — shapes

```
// Each shape has a 'kind' property – that's the discriminant
interface Circle {
  kind: 'circle'; // literal type!
  radius: number;
}

interface Rectangle {
  kind: 'rectangle';
  width: number;
  height: number;
}

interface Triangle {
  kind: 'triangle';
  base: number;
  height: number;
}
```

```
}

type Shape = Circle | Rectangle | Triangle;

function getArea(shape: Shape): number {
  switch (shape.kind) {           // TS narrows by 'kind'
    case 'circle':
      return Math.PI * shape.radius ** 2; // shape: Circle
    case 'rectangle':
      return shape.width * shape.height;  // shape: Rectangle
    case 'triangle':
      return 0.5 * shape.base * shape.height; // shape: Triangle
  }
}
```

Discriminated Unions for React State

This pattern maps perfectly onto React's async state — loading, success, and error are three completely different shapes:

Discriminated union for React fetch state

```
// Three distinct states — each has a unique 'status'
type FetchState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; message: string }

function renderUser(state: FetchState<User>) {
  switch (state.status) {
    case 'idle':
      return 'Not started';
    case 'loading':
      return 'Loading...';
    case 'success':
      return state.data.name; // TS knows .data exists here
    case 'error':
      return state.message;   // TS knows .message exists here
  }
}

// NO accidental access of .data when status is 'loading'!
// TS enforces the correct shape in each branch. Bulletproof.
```

9. Exhaustive Checks — Never Let a Case Slip Through

With discriminated unions, you can make TypeScript ERROR if you forget to handle a case. This is extremely useful when you add a new variant to a union later:

Exhaustive check with `assertNever`

```
function assertNever(value: never): never {
  throw new Error(`Unhandled case: ${JSON.stringify(value)}`);
}

function getArea(shape: Shape): number {
  switch (shape.kind) {
    case 'circle':    return Math.PI * shape.radius ** 2;
    case 'rectangle': return shape.width * shape.height;
    case 'triangle':  return 0.5 * shape.base * shape.height;
    default:          return assertNever(shape);
    // If you add 'Pentagon' to Shape and forget to handle it,
    // TS will error here at compile time. Not at runtime. Magic.
  }
}
```

This pattern saves you in large codebases

When a junior dev (or future you) adds a new status to a union, TypeScript will immediately point to every switch statement that didn't handle it. Zero runtime surprises. This is one of the biggest productivity wins TypeScript gives you over pure JavaScript.

Module Summary

Everything you learned in this module:

- Enums group named constants: `enum Status { Pending, Active }`
- String enums are preferred — more readable than numeric enums
- For simple sets, union types are often better than enums
- Narrowing = proving to TypeScript which specific type you have at a given point
- `typeof` guard: `typeof x === 'string'` — works on primitives
- Truthiness narrowing — `if (x)` removes null/undefined from the type
- `instanceof` guard: `x instanceof MyClass` — works on class instances

- in operator: `'prop' in obj` — narrows by property existence
- Custom type guard: `function isX(val): val is X { ... }`
- Discriminated unions — a shared 'kind' property TS uses to narrow automatically
- Exhaustive check with `assertNever` — compile-time error if you miss a case

> Next: Module 5

Advanced Utility Types

Pick, Omit, ReturnType, Parameters, Readonly, Exclude, Extract and more — the TypeScript standard library that lets you transform and derive types without duplication.